

Technical Test

1. SQL

You have the **customer_courier_chat_messages** table that stores data about individual messages exchanged between customers and couriers via the in-app chat.

You also have access to the **orders** table where you have an `order_id` and `city_code` field.

Your task is to build queries for the following questions (Please add screenshots of the results obtained):

Before answering the questions, I'm going to briefly explain which tools I used and how I prepared the tables to be able to query them.

Firstly, I created two .csv files, one for the *customer_courier_chat_messages* table and another for the *orders* table, using the information provided in the technical-test tables excel file.

Then, I used PostgreSQL, with the tool pgAdmin, where I created a new database. Inside this database I created two new tables, **customer_courier_chat_messages** and **orders**, specifying the name of the columns and the datatypes.

```
5 CREATE TABLE IF NOT EXISTS public.orders
6 (
7     order_id INTEGER PRIMARY KEY,
8     city_code VARCHAR(10)
9 )
```

```

5 CREATE TABLE IF NOT EXISTS public.customer_courier_chat_messages
6 (
7     sender_app_type VARCHAR(50),
8     customer_id INTEGER,
9     from_id INTEGER,
10    to_id INTEGER,
11    chat_started_by_message BOOLEAN,
12    order_id INTEGER,
13    order_stage VARCHAR(50),
14    courier_id INTEGER,
15    message_sent_time VARCHAR(20),
16    CONSTRAINT fk_order
17        FOREIGN KEY (order_id)
18        REFERENCES orders(order_id)
19 )

```

Then, I imported the values of both tables, from pgAdmin, using the .csv files that I previously created.

After importing the values, I changed the datatype of the **message_sent_time** field, from the **customer_courier_chat_messages** table, to **TIMESTAMP**, as I made the import using a **VARCHAR(20)** datatype in this field to avoid potential errors. This way I won't need to use the function **TO_TIMESTAMP** later in each of the following queries.

```

26 ALTER TABLE public.customer_courier_chat_messages
27 ALTER COLUMN message_sent_time TYPE TIMESTAMP
28 USING TO_TIMESTAMP(message_sent_time, 'DD/MM/YYYY HH24:MI');

```

Having explained this, I will now proceed to answer the questions.

1. Show all the columns ordered by **order_id**

Query	Query History
1	SELECT c.*, o.city_code
2	FROM public.customer_courier_chat_messages c
3	JOIN public.orders o
4	ON c.order_id=o.order_id
5	ORDER BY c.order_id;

Data Output Messages Notifications											
	sender_app_type character varying (50)	customer_id integer	from_id integer	to_id integer	chat_started_by_message boolean	order_id integer	order_stage character varying (50)	courier_id integer	message_sent_time timestamp without time zone	city_code character varying (10)	
1	Courier Android	122	87	122	true	38	ADDRESS_DELIVERY	87	2022-08-09 07:55:00	BCN	
2	Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:00:00	OPO	
3	Courier IOS	23	21	23	false	134	ARRIVING	21	2022-08-07 10:01:00	OPO	
4	Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:02:00	OPO	
5	Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:02:00	BCN	
6	Courier IOS	99	21	99	false	555	ARRIVING	21	2022-08-09 08:01:00	BCN	
7	Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:00:00	BCN	
8	Courier Android	43	75	43	true	875	ADDRESS_DELIVERY	75	2022-08-07 14:50:00	VAL	
9	Customer Android	43	43	75	false	875	PICKING_UP	75	2022-08-07 14:51:00	VAL	
10	Courier Android	43	75	43	false	875	ARRIVING	75	2022-08-07 14:53:00	VAL	
11	Customer Android	43	43	75	false	875	PICKING_UP	75	2022-08-07 14:55:00	VAL	

2. Show all the columns ordered by **city_code**

Query Query History

```

1 SELECT c.*, o.city_code
2 FROM public.customer_courier_chat_messages c
3 JOIN public.orders o
4 ON c.order_id=o.order_id
5 ORDER BY o.city_code;
```

Data Output Messages Notifications											
	sender_app_type character varying (50)	customer_id integer	from_id integer	to_id integer	chat_started_by_message boolean	order_id integer	order_stage character varying (50)	courier_id integer	message_sent_time timestamp without time zone	city_code character varying (10)	
1	Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:02:00	BCN	
2	Courier IOS	99	21	99	false	555	ARRIVING	21	2022-08-09 08:01:00	BCN	
3	Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:00:00	BCN	
4	Courier Android	122	87	122	true	38	ADDRESS_DELIVERY	87	2022-08-09 07:55:00	BCN	
5	Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:00:00	OPO	
6	Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:02:00	OPO	
7	Courier IOS	23	21	23	false	134	ARRIVING	21	2022-08-07 10:01:00	OPO	
8	Courier Android	43	75	43	true	875	ADDRESS_DELIVERY	75	2022-08-07 14:50:00	VAL	
9	Customer Android	43	43	75	false	875	PICKING_UP	75	2022-08-07 14:55:00	VAL	
10	Courier Android	43	75	43	false	875	ARRIVING	75	2022-08-07 14:53:00	VAL	
11	Customer Android	43	43	75	false	875	PICKING_UP	75	2022-08-07 14:51:00	VAL	

3. Show the first message (row) sender (courier or customer)

Query Query History

```

1 SELECT c.*
2 FROM public.customer_courier_chat_messages c
3 ORDER BY c.message_sent_time ASC
4 LIMIT 1;
```

Data Output Messages Notifications											
	sender_app_type character varying (50)	customer_id integer	from_id integer	to_id integer	chat_started_by_message boolean	order_id integer	order_stage character varying (50)	courier_id integer	message_sent_time timestamp without time zone		
1	Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:00:00		

4. Show the number of messages sent by customer and order_id

Query Query History

```

1 SELECT
2     order_id,
3     COUNT(*) AS customer_sent_messages
4 FROM public.customer_courier_chat_messages
5 WHERE sender_app_type LIKE 'Customer%'
6 GROUP BY order_id;

```

Data Output Messages Notifications

	order_id integer	customer_sent_messages bigint
1	134	2
2	555	2
3	875	2

5. Show the first message (row) in the conversation by order_id

Query Query History

```

1 WITH ranked_messages AS(
2     SELECT *,
3     ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY message_sent_time ASC) AS ranking
4 FROM public.customer_courier_chat_messages
5 )
6
7 SELECT *
8 FROM ranked_messages
9 WHERE ranking = 1;

```

Data Output Messages Notifications

	sender_app_type character varying (50)	customer_id integer	from_id integer	to_id integer	chat_started_by_message boolean	order_id integer	order_stage character varying (50)	courier_id integer	message_sent_time timestamp without time zone	ranking bigint
1	Courier Android	122	87	122	true	38	ADDRESS_DELIVERY	87	2022-08-09 07:55:00	1
2	Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:00:00	1
3	Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:00:00	1
4	Courier Android	43	75	43	true	875	ADDRESS_DELIVERY	75	2022-08-07 14:50:00	1

Showing rows: 1 to 4 Page No: 1 of 1

6. Show the last message (row) in the conversation by order_id

Query

Query History

```
1 WITH ranked_messages AS(
2     SELECT *,
3     ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY message_sent_time DESC) AS ranking
4 FROM public.customer_courier_chat_messages
5 )
6
7 SELECT *
8 FROM ranked_messages
9 WHERE ranking = 1;
```

Data Output

Messages

Notifications

Showing rows: 1 to 4

Page No: 1

of 1

	sender_app_type character varying (50)	customer_id integer	from_id integer	to_id integer	chat_started_by_message boolean	order_id integer	order_stage character varying (50)	courier_id integer	message_sent_time timestamp without time zone	ranking bigint
1	Courier Android	122	87	122	true	38	ADDRESS_DELIVERY	87	2022-08-09 07:55:00	1
2	Customer IOS	23	23	21	false	134	PICKING_UP	21	2022-08-07 10:02:00	1
3	Customer IOS	99	99	21	false	555	PICKING_UP	21	2022-08-09 08:02:00	1
4	Customer Android	43	43	75	false	875	PICKING_UP	75	2022-08-07 14:55:00	1

7. Show the time (in secs) elapsed until the first message was responded by order_id

Query

Query History

```
1 WITH first_messages AS (
2     SELECT order_id, sender_app_type, message_sent_time AS msg_time,
3         ROW_NUMBER() OVER(PARTITION BY order_id ORDER BY message_sent_time ASC) as ranking
4     FROM customer_courier_chat_messages
5 ),
6 first_reply AS (
7     SELECT m.order_id, m.message_sent_time AS reply_time
8     FROM customer_courier_chat_messages m
9     JOIN first_messages fm ON m.order_id = fm.order_id AND fm.ranking = 1
10    WHERE m.sender_app_type != fm.sender_app_type
11          AND m.message_sent_time > fm.msg_time
12 )
13 SELECT fm.order_id,
14     ROUND(EXTRACT(EPOCH FROM (MIN(fr.reply_time) - fm.msg_time)), 2) AS response_delay_seconds
15 FROM first_messages fm
16 JOIN first_reply fr ON fm.order_id = fr.order_id
17 WHERE fm.ranking = 1
18 GROUP BY fm.order_id, fm.msg_time;
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 3

	order_id integer	response_delay_seconds numeric
1	134	60.00
2	555	60.00
3	875	60.00

8. Build a query that aggregates individual messages into conversations. The query result should be used to create a table `customer_courier_conversations` (take into consideration that a conversation is unique per order).

a. The required fields are the following:

i. `order_id`

ii. `city_code`

iii. `first_courier_message`: Timestamp of the first courier message

iv. `first_customer_message`: Timestamp of the first customer message

v. `num_messages_courier`: Number of messages sent by courier

vi. `num_messages_customer`: Number of messages sent by customer

vii. `first_message_by`: The first message sender (courier or customer)

viii. `conversation_started_at`: Timestamp of the first message in the conversation

ix. `first_responsetime_delay_seconds`: Time (in secs) elapsed until the first message was responded

x. `last_message_time`: Timestamp of the last message sent

xi. `last_message_order_stage`: The stage of the order when the last message was sent

```

1  WITH courier_messages AS (
2      SELECT
3          order_id,
4          MIN(message_sent_time) AS cour_first_timest,
5          COUNT(*) AS cour_num_mess
6      FROM customer_courier_chat_messages
7      WHERE sender_app_type LIKE 'Courier%'
8      GROUP BY order_id
9  ),
10 customer_messages AS (
11     SELECT
12         order_id,
13         MIN(message_sent_time) AS cust_first_timest,
14         COUNT(*) AS cust_num_mess
15     FROM customer_courier_chat_messages
16     WHERE sender_app_type LIKE 'Customer%'
17     GROUP BY order_id
18 ),
19 first_last_messages AS (
20     SELECT DISTINCT order_id,
21         CASE WHEN
22             FIRST_VALUE(sender_app_type) OVER(PARTITION BY order_id ORDER BY message_sent_time ASC) LIKE 'Courier%' THEN 'Courier'
23             ELSE 'Customer'
24         END AS first_sender,
25         FIRST_VALUE(message_sent_time) OVER(PARTITION BY order_id ORDER BY message_sent_time ASC) AS first_timestamp,
26         FIRST_VALUE(message_sent_time) OVER(PARTITION BY order_id ORDER BY message_sent_time DESC) AS last_timestamp,
27         FIRST_VALUE(order_stage) OVER(PARTITION BY order_id ORDER BY message_sent_time DESC) AS last_order_stage
28     FROM customer_courier_chat_messages
29 )
30 SELECT
31     o.order_id,
32     o.city_code,
33     come.cour_first_timest AS first_courier_message,
34     cume.cust_first_timest AS first_customer_message,
35     COALESCE(come.cour_num_mess, 0) AS num_messages_courier,
36     COALESCE(cume.cust_num_mess, 0) AS num_messages_customer,
37     flm.first_sender AS first_message_by,
38     flm.first_timestamp AS conversation_started_at,
39     ROUND(ABS(EXTRACT(EPOCH FROM (come.cour_first_timest - cume.cust_first_timest))), 2) AS first_responsetime_delay_seconds,
40     flm.last_timestamp AS last_message_time,
41     flm.last_order_stage AS last_message_order_stage
42 FROM orders o
43 LEFT JOIN courier_messages come
44 ON o.order_id = come.order_id
45 LEFT JOIN customer_messages cume
46 ON o.order_id = cume.order_id
47 LEFT JOIN first_last_messages flm
48 ON o.order_id=flm.order_id;

```

Data Output Messages Notifications

order_id (PK) integer	city_code character varying (10)	first_courier_message timestamp without time zone	first_customer_message timestamp without time zone	num_messages_courier bigint	num_messages_customer bigint	first_message_by text	conversation_started_at timestamp without time zone	first_responsetime_delay_seconds numeric	last_message_time timestamp without time zone	last_message_order_stage character varying
1	875 VAL	2022-08-07 14:50:00	2022-08-07 14:51:00	2	2	Courier	2022-08-07 14:50:00	60.00	2022-08-07 14:55:00	PICKING_UP
2	134 OPO	2022-08-07 10:01:00	2022-08-07 10:00:00	1	2	Customer	2022-08-07 10:00:00	60.00	2022-08-07 10:02:00	PICKING_UP
3	555 BCN	2022-08-09 08:01:00	2022-08-09 08:00:00	1	2	Customer	2022-08-09 08:00:00	60.00	2022-08-09 08:02:00	PICKING_UP
4	38 BCN	2022-08-09 07:55:00	[null]	1	0	Courier	2022-08-09 07:55:00	[null]	2022-08-09 07:55:00	ADDRESS_DELIVERY